

BSc (Hons) in **Computer Science**

Faculty of Computing

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026

Practical 07

> **Module Objective:** In previous weeks, your Drow Ranger agent used A* Search to find a physical path to the goal. However, it blindly assumed every non-wall tile was safe. Today, we implement a **Knowledge Base (KB)** and a **Forward Chaining Inference Engine**. Your agent must now mathematically prove a cell is logically *Feasible* before taking a step, using real-time rules about map vision and enemy threats.

> **Before you start:** This lab continues the project from Labs 1–4. Open the same project and check that `SearchAgent.astar_search()` in `agent.py` works.
>
> - Parts 1, 2 & 4: new files (`logic_engine.py` , `test_logic.py`) in the project folder.
> - Part 3: edit your existing `agent.py` .
> - **Create a separate branch `lab-07`** and start your work there. When finished, **commit your code and push the branch**, as you did in Labs 1–4.
> - **Upload this worksheet's answered PDF to the submission portal.**

Part 1: The Logic Engine Architecture

Step 1.1: Building the Knowledge Base (KB)

> Instead of hardcoding hundreds of nested `if-else` statements in our agent's movement logic, we will build a declarative Knowledge Base. You need to design a Python class that stores **Facts** (current percepts) and **Rules** (Horn Clauses).

1. Create a new file named `logic_engine.py` .
2. Implement a `KnowledgeBase` class based on the following structural requirements:

```
CLASS KnowledgeBase:
   // Attributes
```

```
DECLARE facts AS Set                    // To store unique string fact
DECLARE rules AS List                   // To store rules as Tuples: (

// Methods
FUNCTION tell_fact(fact_string):
    ADD fact_string TO facts

FUNCTION tell_rule(premise_list, conclusion_string):
    APPEND Tuple(premise_list, conclusion_string) TO rules

FUNCTION clear_facts():
    EMPTY the facts Set
```

Part 2: Implementing Forward Chaining

Step 2.1: The Inference Algorithm

> The agent needs an Inference Engine to deduce new information from its raw sensors. Translate the following Data-Driven **Forward Chaining** algorithm into a Python method inside your `KnowledgeBase` class.

1. Write the `forward_chain()` method.
2. Ensure you use a `while` loop that continues to iterate over the rules until a full pass results in absolutely no new facts being deduced.

```
FUNCTION forward_chain():
   DECLARE new_facts_added = TRUE

   WHILE new facts added == TRUE:
       new_facts_added = FALSE

       FOR EACH (premises, conclusion) IN rules:
           IF conclusion IS NOT IN facts:

               // Modus Ponens Check
               IF ALL premises ARE IN facts:
```

```
ADD conclusion TO facts
new_facts_added = TRUE
```

Part 3: Hooking Logic into the Grid Game

Step 3.1: Defining the Game Constraints

> Now we translate the game's safety constraints into our logic engine using Propositional Logic.

1. In `agent.py`, add `from logic_engine import KnowledgeBase`, then create the KB in `SearchAgent.__init__`: `self.kb = KnowledgeBase()`.
2. Use your `tell_rule` method to define the following safety rules:
   Rule 1: `TargetVisible ∧ HasDust ⇒ SafeToEngage`
   Rule 2: `SafeToEngage ∧ BloodseekerMissing ⇒ Retreat`

The game does not produce these facts, so you supply them per tile as a dictionary: `{(x, y): [facts]}`. A tile where `Retreat` is deduced is unsafe to enter.

Step 3.2: Validating Feasibility in A*

> Modify your existing A* pathfinding algorithm to consult the Knowledge Base before expanding a node.

1. Add a parameter `tile_facts=None` to `astar_search()` (treat `None` as `{}`).
2. Locate the neighbor-generation loop in your A* code (where you normally check for physical walls).
3. Before adding a neighbor to the `open_list`, clear the KB facts.
4. Feed that tile's facts into the KB: `for fact in tile_facts.get(successor, []): self.kb.tell_fact(fact)`.
5. Run `self.kb.forward_chain()`.
6. If `'Retreat'` is deduced and exists in `self.kb.facts`, mark the tile as **Infeasible**. Skip it (do not add it to the open list), even if it is physically reachable.

Step 3.3: Run It in the Grid

1. In `sense_and_act()`, define sample facts and pass them to A*:

```
tile facts = {
    (1, 0): ['TargetVisible', 'HasDust', 'BloodseekerMissing'],  # dedu
    (0, 1): ['TargetVisible', 'HasDust'],                        # Safe
}
self.plan = self.astar_search(position, nearest_food, walls, grid_size,
```

2. Run `visual_grid_game.py`. The agent should never step onto `(1, 0)`.

Part 4: Self-Evaluation Test Cases

4.1 Automated Validation

Create a file named `test_logic.py`. Copy and run the following Python assert statements to verify your Forward Chaining engine works correctly mathematically before you attempt to integrate it into the visual grid.

```
from logic_engine import KnowledgeBase

def test_forward_chaining():
    kb = KnowledgeBase()

    # Add Domain Rules
    kb.tell_rule(['TargetVisible', 'HasDust'], 'SafeToEngage')
    kb.tell_rule(['SafeToEngage', 'BloodseekerMissing'], 'Retreat')

    # Test Case 1: Safe Engagement
    kb.clear_facts()
    kb.tell_fact('TargetVisible')
    kb.tell_fact('HasDust')
    kb.forward_chain()
    assert 'SafeToEngage' in kb.facts, "Test 1 Failed: Should deduce Sa
    assert 'Retreat' not in kb.facts, "Test 1 Failed: Should NOT deduce

    # Test Case 2: Unsafe Engagement (Bloodseeker Missing)
    kb.clear_facts()
```

```
        kb.tell_fact('TargetVisible')
        kb.tell_fact('HasDust')
        kb.tell fact('BloodseekerMissing')
        kb.forward_chain()
        assert 'Retreat' in kb.facts, "Test 2 Failed: Should deduce Retreat

        print("✅ All Logic Engine Test Cases Passed!")

    if __name__ == "__main__":
        test_forward_chaining()
```

Part 5: Theory-to-Implementation Mapping

> Based on the code you just wrote and the lecture on Logical Reasoning, complete the following analytical questions to explain *how* your code implements the theory.

1. Reachability vs. Feasibility (Apply)
In Step 3.2, you modified the A* algorithm's neighbor-generation loop. How does your new code differentiate between checking if a node is "Reachable" (like a wall collision) versus checking if it is "Feasible"? Explain how the Knowledge Base enforces feasibility.

The code checks a neighbour in two separate stages:

1)Reachable: get_successors() only returns tiles that are inside the grid and are not walls. This is a physical check: can the agent move there at all?

2)Feasible: for every reachable neighbour, the code clears the KB, adds that tile's facts with tell_fact(), and runs forward_chain(). If 'Retreat' is deduced, the code skips the tile with continue, so it is never added to the frontier.

A tile can be reachable but still infeasible. In the test run, (1, 0) had no wall, but the KB deduced Retreat, so A* went around it. This made the path 18 moves instead of 14.

2. Declarative vs. Procedural Paradigms (Analyze)
Why did we build a `KnowledgeBase` class with a `forward_chain()` loop instead of simply writing `if 'TargetVisible' in percepts and 'HasDust' in`

`percepts:` directly inside the agent's movement code? What happens to the agent's code complexity if we add 50 new game rules?

The KnowledgeBase keeps the rules (what is true) separate from the inference engine (how to reason). Rules are just data added with tell_rule(). forward_chain() stays the same no matter how many rules there are, and it also handles chained rules automatically. For example, SafeToEngage leads on to Retreat.

With hard-coded if statements, every rule and every chain of rules would have to be written out by hand inside the movement code. Adding 50 new rules would create long, deeply nested if-else blocks that are hard to read, test, and change. With the KB, adding 50 rules just means 50 more tell_rule() lines, and the A* code does not change at all.

3. Modus Ponens & Horn Clauses (Understand)

The rule `['TargetVisible', 'HasDust'] ⇒ 'SafeToEngage'` is formally known as a Horn Clause. Briefly explain how your Python implementation of the `forward_chain()` `while` loop acts as a mathematical execution of the *Modus Ponens* inference rule.

Modus Ponens says: if P -> Q is true and P is true, then Q is true. A Horn clause such as TargetVisible ∧ HasDust -> SafeToEngage has several premises joined by AND and exactly one conclusion.

In forward_chain(), for each rule (premises, conclusion):

- all(p in self.facts for p in premises) checks that P (every premise) is true.
- self.facts.add(conclusion) then concludes Q.

The while loop repeats this until a full pass adds no new facts. This means conclusions can be used as premises for later rules, as when SafeToEngage lets the KB deduce Retreat.

Finalize and Lock Lab 07 Implementation



FACULTY OF COMPUTING